

VILNIAUS UNIVERSITETAS
MATEMATIKOS IR INFORMATIKOS FAKULTETAS
PROGRAMŲ SISTEMŲ STUDIJŲ PROGRAMA

**Hibridinio genetinio paieškos algoritmo transporto
maršrutų optimizavimo uždaviniams spręsti
lygiagretinimas**

**Parallelization of Hybrid Genetic Search Algorithm for Solving
Vehicle Routing Problem**

Kursinis darbas

Atliko: 4 kurso 1 grupės studentas
Domantas Keturakis

Darbo vadovas: Doc., Dr. Algirdas Lančinskas

Vilnius – 2025

Turinys

Terminai	3
Įvadas	4
Uždavinio apibrėžimas	6
HGS algoritmo veikimas	6
Literatūros analizė	8
Lygiagretinimas	9
Metodika	10
Rezultatai	10
Išvados	11
Priedai	12
Šaltiniai	15

Terminai

- Populiacija – Rinkinys invididų.
- Individas – Užduoties sprendinys t.y. rinkinys maršrutų.
- VRP – Martšrutų optimizavimo uždavinys (*angl. Vechicle Routing Problem*).
- CVRP – *angl. Capacitated Vehicle Routing Problem*. Kiekviena transporto priemonė turi maksimalią siuntų talpą.
- VRPTW – *angl. VRP with Time Windows*.
- GVRP – *angl. Generalized VRP*. Klientai grupuojami į klusterius. Tik vienas klientas iš viso klusterio turi būti aplankytas.
- CluVRP – *angl. Clustered VRP*. Klientai grupuojami į klusterius. Visi klientai klusteryje turi būti aplankyti prieš važiuojant į kitą klusterį.
- SoftCluVRP – *angl. Clustered VRP*. Klientai grupuojami į klusterius. CluVRP variantas, kuriame į klusterį leidžiama aplankyti kelis kartus.
- MDVRP – *angl. Multidepot VRP*.
- PVRP – *angl. Periodic VRP*. Pridedama laiko dimescija, sprendinys susidaro iš kelių maršrutų rinkinių atitinkačius dienas, kuriomis bus aplankomi klientai.
- MDPVRP – *angl. Multidepot Periodic VRP*. MDVRP ir PVRP kombinacija.
- CVRPPD – *angl. CVRP Pickup and Delivery*. CVRP ir VRPPD kombinacija.

Įvadas

VRP – Transporto maršrutų optimizavimo uždavinys (*angl. Vehicle Routing Problem*) yra uždavinys, kurio tikslas yra surasti kuo optimaliausią maršrutų rinkinį Pav. 1. Optimaliai parinkti maršrutai gali lemti kiek klientų įmanoma aplankyti per nustatytą laiką, sumažinti transporto kaštus. Pirmą kartą ši problema aprašyta [DR59], kur autorius aprašė algoritmą, kuris suranda optimalius maršrutus tarp kuro depo ir degalinių. Tai yra modernios logistikos optimizavimo uždavinys – optimaliai parinkti maršrutai gali lemti mažesnius kainos ir pristatymo laiko kaštus.

Kur keliaujančio pardavėjo uždavinyje pagrindinė užduotis yra surasti optimaliausią kelią vienam keliautojui – pardavėjui, VRP sprendiniai susidaro iš kelių keliautojų – literatūroje dažnai tiesiogiai vadinama transporto priemonėmis.

Nors egzistuoja įrankiai, kurie pasiteklia tikslūs metodai (pavyzdžiui „Google or-tools“ [PF25]), kadangi šis uždavinys priklauso *NP-Hard* sudėtingumo klasei, visgi dominuoja heuristikomis ir metaheuristikomis grįsti algoritmai, nes šie beveik optimalius sprendinius suranda per greitesnį laiko tarpą sunaudodami mažiau resursų. Tikslūs metodai su ypač dideliais kiekiais duomenų tampa nepraktiški. Metaheuristiciniai algoritmai išsiskiria šioje uždavinių klasėje kaip efektyviausi, pasižymintys žemu algoritmo vykdymo laiku ir aukšta uždavinių rezultatų kokybe.

Praktikoje taikomos VRP variacijos (CVRP, VRPTW, MDPVRP, PVRP ir kt.) įveda papildomus apribojimus maršrutų ilgiui, transporto priemonių panaudojimo laikui ir talpai, ar prideda papildomas sąlygas:

- naudojamos transporto priemonės turi limituotą talpą (CVRP).
- visi klientai turi gali būti aplankyti tik specifinėmis darbo valandomis (VRPTW)
- keli depai iš kurių galima pradėti maršrutą (MDVRP),
- maršrutai planuojami per kelias dienas, t.y. vieni klientai gali būti aplankyti vieną dieną, o kiti kitą. (Periodic VRP).
- kt.

Šis darbas atsižvelgia tik į CVRP uždavinį.

Šiai problemai egzistuoja heuristiniai ir metaheuristiniai (Aukštesnio lygio strategija/karkasas, kuris dirguoja, kurias heuristikas pritaikyti, kad efektyviau atrasti sprendinius) algoritmai. Keli dominuojantys pavyzdžiai [Ada+24]

- „Adaptive Large Neighborhood Search“ ir „Hybrid Adaptive Large Neighborhood Search“
- „Hybrid Genetic Search (HGS)“
- „Simulated Annealing Algorithm (SAA)“
- „Ant colony optimization (ACO)“

Hibridinis genetinis paieškos (*angl. Hybrid Genetic Search – HGS*) – yra vienas iš efektyviausių genetinių metaheuristicinių algoritmų [Pet+23]. Šis algoritmas ir vėlesnės pagerintos versijos išlieka etalonas daugeliui VRP variantų, „DIMACS“ konkurse [DIM22] parodęs geriausius rezultatus VRPTW uždavinyje [Koo+22], ir kurio modifikuotas variantas [Jia+22] pasirodė geriausiai CVRP uždavinyje. Šis algoritmas yra pritaikytas CVRP, VRPTW, GVRP [Lat25a], CluVRP, SoftCluVRP [Lat25b].

Šio **darbo tikslas** – išlygiagretinti hibridinio genetinio paieškos algoritmą, skirtą transporto maršrutų optimizavimo uždaviniams spręsti, siekiant sumažinti vykdymo laiką neprarandant ar net pagerinant sprendinių kokybę.

Uždaviniai:

1. Išsirinkti duomenų rinkinį pagal, kurį galima būtų testuoti/analizuoti sprendinius.
2. Išanalizuoti, kaip veikia HGS algoritmas
3. Atrinkti paralelizuojamas dalis, ar dalis, kurias galima pakeisti paralelizuojamomis.
4. Palyginti rezultatus su kitais state-of-the-art algoritmais

Uždavinio apibrėžimas

CVRP nagrinėjamas grafas $G = (V, E)$, kuriame $v_0 \in V$ žymi depą, kuris turi m transporto priemonių, o likusios viršūnės $\{v_1, \dots, v_{|V|}\}$ atitinka klientus, kuriuos reikia aplankyti. Kiekviena briauna $(i, j) \in E$ reiškia galimybę keliauti tarp vietų i ir j su kaina $c_{i,j}$ – atstumas tarp vietų i ir j . CVRP reikia surasti sprendinį, kuriame panaudotos ne daugiau kaip K transporto priemonių, prasidedančių ir pasibaigiančių depe, taip, kad kiekvienas klientas būtų aplankytas vieną kartą ir bendras klientų paklausos dydis bet kuriame maršrute neviršytų transporto priemonės talpos Q , o bendras transporto priemonių nuvažiuotas atstumas – kaina Pav. 1 kiek įmanoma mažesnis.

$$\text{Sprendinio kaina} = \sum_{k=1}^K \sum_{i=0}^{|V|} \sum_{j=0}^{|V|} c_{i,j} x_{i,j,k}$$

$c_{i,j}$ = atstumas nuo kliento i iki kliento j

$x_{i,j,k} = 1$ Indikatorinė funkcija, kuri

lygi 1, jei transporto priemonė k ($1 \leq k \leq K$)

keliauja nuo kliento i iki kliento j ,

lygi 0 priešingu atveju

Pav. 1: Sprendinio kainos apibrėžimas

Lyginant transporto maršrutų optimizavimo uždavinio sprendinius taip pat naudojamas tarpas (*angl. gap*), kuris nusako atstumą nuo geriausio sprendinio išreikštas procentais Pav. 2.

$$\text{Tarpas} = \left(\frac{Z_s - Z_{\text{BKS}}}{Z_{\text{BKS}}} \right) \cdot 100\%$$

Z_s = Pasirinkto algoritmo sprendinio kaina

Z_{BKS} = Geriausio sprendinio kaina

Pav. 2: Tarpo apibrėžimas

HGS algoritmo veikimas

Pirma aprašytas "A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems" [Vid+12] (2012) skirtas spręsti MDPVRP. Patobulintas per daugelį iteracijų: [Vid+14], [Vid+16], [Vid17], [Vid+21], [Vid22] ir pritaikytas CVRP.

- Beyond a simple reimplementation of the original algorithm, HGS- CVRP takes advantage of several lessons learned from the past decade of VRP studies: it relies on simple data structures to avoid move reevaluations and uses the optimal linear-time Split algorithm of Vidal (2016). Moreover, its specialization to the CVRP permits significant methodological simplifications. In particular, it does not rely on the visit-pattern improvement (PI) operator (Vidal et al., 2012) originally designed for VRPs with multiple periods, and uses instead a new neighborhood called Swap*.
— Thibaut Vidal [Vid22]
- In HGS-CVRP, we rely on the efficient linear-time Split algorithm introduced by Vidal (2016) (autorius papildymas: [Vid16]) after each crossover operation.
— Thibaut Vidal [Vid22]

Genetiniai algoritmai imituoja evoliucijos procesą. Populiacija yra aibė, kurią sudaro individai (t.y. užduoties sprendiniai). Šie algoritmai naudoja įvairius kryžminimo operatorius, kurie iš kelių individų populiacijoje sukuria naują, mutuotą individą ir prideda prie populiacijos. Prastos kokybės ir panašūs individai (sprendiniai) vykdymo eigoje yra pašalinami iš populiacijos.

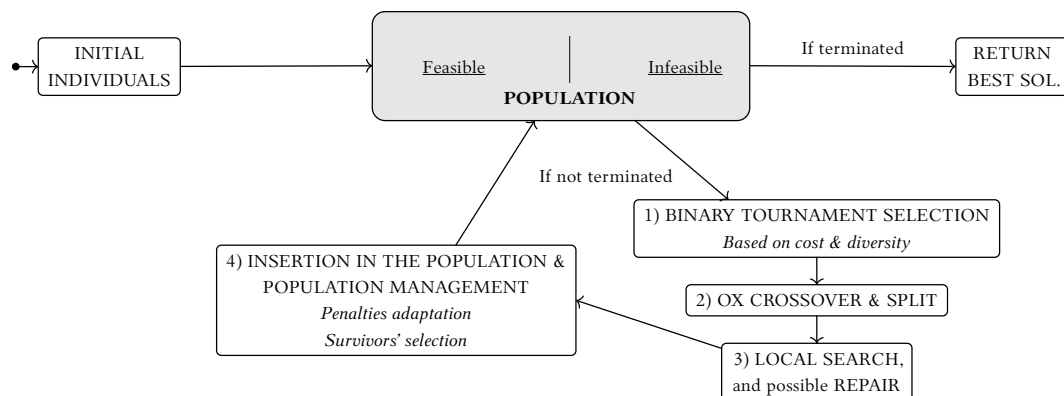
Genetinis hybridinis paieškos algoritmas prie genetinio komponento prideda pagerinimo žingsnį (vietinę paiešką (*angl. local search*)), kuri po kryžminimo žinsnio yra pritaikoma naujam individui, kad pagerinti gautą individo kokybę.

Vietinėje paieškoje taikomos *relocate*, *2-opt*, *2-opt**, bei *swap** heuristikos.

Palaikant neįvykdomus (*angl. infeasible*) ir įvykdomus (*angl. feasible*) sprendinius, palaikoma populiacijos įvairovė (*angl. diversity*), kuri leidžia išvengti lokalios minimos (*angl. local minima*) iteruojant per sprendinius.

- 1 Sugeneruoti pradinę populiaciją ir pagerinti ją vietine paieška
- 2 **Kol** iteracijų be pagerėjimo skaičius mažesnis už ribą ir $t < T_{\max}$ atlikti
- 3 Pasirinkti tėvinius individus (dvejietinis turnyras (*angl. binary tournament*))
- 4 Atlikti kryžminimą (*angl. crossover*)
- 5 Išmokyti naują individą (vietinė paieška)
- 6 Įterpti išmokytą individą į atitinkamą subpopuliaciją
- 7 **Jeigu** individas neįvykdomas:
- 8 Su 50% tikimybe bandyti sutaisyti individą ir įtraukti į atitinkamą subpopuliaciją
- 9 **Jeigu** pasiektas maksimalus aibės dydis:
- 10 pašalinti blogiausius ir neįvairius individus iš populiacijos
- 11 Patikslinti baudos parametrus (*angl. penalty parameters*)
- 12 Grąžinti geriausią įvykdomą individą

Pav. 3: HGS algoritmo pseudokodas [Vid+12]



Pav. 4: HGS veikimas [Vid22]

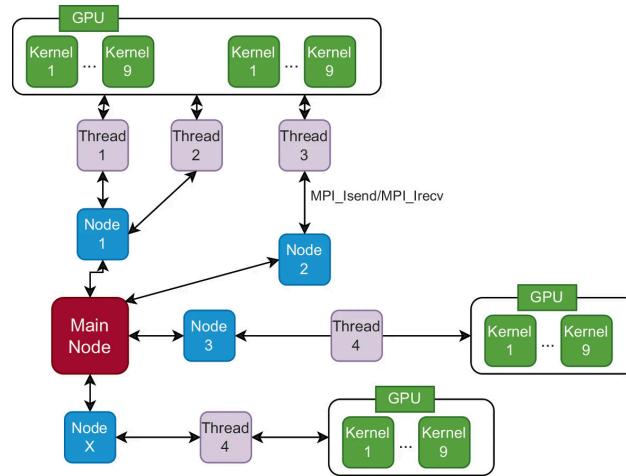
TODO: Išversti į lietuvių k.???

Literatūros analizė

[LHW25] lygiagretinimui vietinės paieškos algoritmą išreiškia tenzorių operatoriais, tai leidžia HGS vykdymą perkelti ant GPU. Taip pagreitintas vietinės paieškos operatorius. Tačiau [LHW25] pasiūlytas metodas nėra pritaikomas HGS su swap*. „Dabartinė sprendinių reprezentacija per tensorius neleidžia lengvai įgyvendinti apkarpymo strategijų kaimynysčių sumažinimo technikų, kurie dažnai yra naudojami vietinės paieškos grįžtais algoritmais.“¹

[SND23] HGS pritaiko CVRPPD, perkelia tėvų pasirinkimo (*angl. selection*), kryžminimo (*angl. crossover*) ir taisymo (*angl. repair*) žingsnius į atskiras gijas. Kiekviena gija papildomai atlieka vietinę paiešką (*2-opt*, *relocate*, *swap*) pasitelkiant GPU, tačiau nepasitelkia *swap** heuristika, kuri pagal [Vid22] padeda surasti aukštesnės kokybės sprendinius.

Priešintai nei dauguma implementacijų [Mun+23] naudoja grafų duomenų struktūras, panaudojami tik *2-opt* ir arčiausio kaimyno heuristikos (*angl. nearest-neighbor*). Šios heuristikos pritaikytos vykdymui GPU aplinkoje naudojant CUDA.

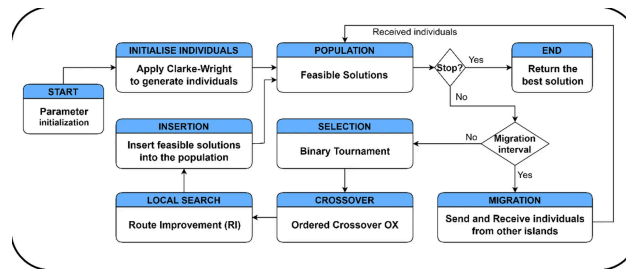


Pav. 5: HGS lygiagretintas [SND23]

[YT21] pasitelkia GPU lygiagretinimui. Kiekvienam maršrutui skiriama atskira GPU gija, kuri vykdo vietinės paieškos žingsnį naudojant GPU. Šiuo metodu pilnas resursų išnaudojimas priklauso nuo to ar sukurtų maršrutų skaičius sutampa su gijų skaičiumi. Atvejai, kai vietinės paieškos žingsniai modifikuoja kitų maršrutų sprendinį, gijos įrašo savo sprendinius į atskirą masyvą, kuris vėliau yra redukuojamas į vieną sprendinį, pasirenkant geriausią sprendinį. Analogiškai, vėlesnėme žingsnyje kiekvienam klientui priskiriama gija. Kiekviena gija apskaičiuoja pagerėjimą ar pablogėjimą apsiikeistus vietą maršrute su kitu klientu.

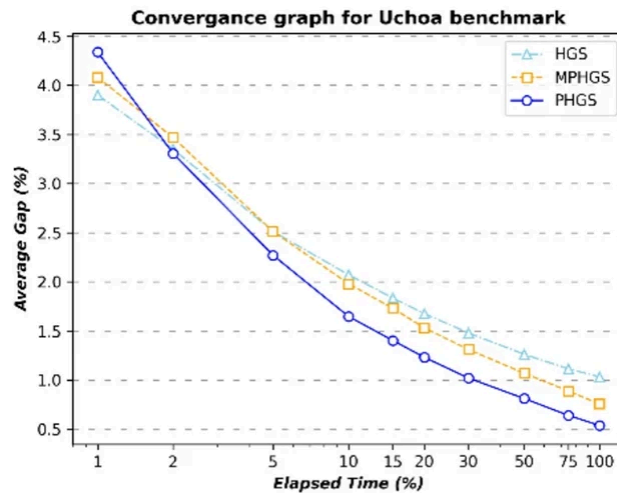
[Jam+25] aprašo *PHGS* (*angl. Parallel Hybrid Genetic Search*), kur kombinuoja HGS su salų modeliu, aprašytu [Rez+24], kur kiekviena gija, vykdo tą patį HGS algoritmą, pridamas individų migracijos žingsnis, kuris leidžia keistis sprendiniais tarp gijų.

¹the current design of the tensor representation of solutions doesn't support easy implementation of pruning strategies and neighborhood reduction techniques that are often used in local search-based routing algorithms.



Pav. 6: HGS su salų modeliu [Jam+25]

PHGS rodo vos ne du kartus geresnius rezultatus galutiniame laiko momente.



Pav. 7: Palyginimas tarp vidutinio tarpo (Y ašis) ir vykdymo laiko (X ašis) [Jam+25].

Lygiagretinimas

Paimta [Vid22] HGS algoritmo implementacija², kuri naudojama kaip pagrindas lygiagretinimui.

Daugiausiai laiko užima vietinės paieškos žingsnis [Jam+25], šio autoriaus atliktais matavimais vietinė paieška užima 85% vykdymo laiko. Todėl siekiant sumažinti viso HGS algoritmo vykdymo laiką šį žingsnį yra labiausiai verta lygiagretinti.

Lygiagretinimas įgyvendintas kiekvienai gijai, atliekant vietinę paiešką. Lygiagretinta HGS-CVRP versija pavaizduota Pav. 8. Nuosekliai veikiančioje sekcijoje parenkami skirtingi tėviniai individai, kryžminimo metu sukuriama kiekvienai gijai. Tada kiekviena gija lygiagrečiai atlieka vietinės paieškos žingsnį. Vėliau, kai kiekviena gija atlieka šį žingsnį, individai yra pridedami į visų populiaciją.

TODO: praplėsti ir padaryti diagramą

²Nuolatinė repozitoriją nuoroda <https://github.com/vidalt/HGS-CVRP/tree/1a927955cd2861a29d978f0d359d6e647db9319c>

- 1 Sugeneruoti pradinę populiaciją ir pagerinti ją vietine paieška
- 2 **Kol** iteracijų be pagerėjimo skaičius mažesnis už ribą ir $t < T_{\max}$ atlikti
- 3 Pasirinkti $N/3 * 2$ tėvinius individus (dvejietinis turnyras (*angl. binary tournament*))
- 4 Atlikti kryžminimą (*angl. crossover*) N kartų
- 5 (Kiekvienoje gijoje) išmokyti naują individą
- 6 Įterpti išmokytą individą į atitinkamą subpopuliaciją
- 7 **Jeigu** individas neįvykdomas:
- 8 (Kiekvienoje gijoje) su 50% tikimybe bandyti sutaisyti individą ir įtraukti į atitinkamą subpopuliaciją
- 9 **Jeigu** pasiektas maksimalus aibės dydis:
- 10 pašalinti blogiausius ir neįvairius individus iš populiacijos
- 11 Patikslinti baudos parametrus (*angl. penalty parameters*)
- 12 Grąžinti geriausią įvykdomą individą

Pav. 8: HGS algoritmo pseudokodas [Vid+12]

Metodika

Dėl rezultatų palyginamumo pasirinkta naudoti [Vid22] aprašytus duomenų rinkinius ([Uch+17]) ir metodiką.

„[Vid22].“⁴

HGS algoritmas sustoja tik, kai pasiekiamas iteracijų skaičius be pagerėjimo ar veikimo laikas viršija tuos nustatytus pagal parinkus parametrus. Parinkus pakankamai aukštus parametrus algoritmo veikimo laikas gali tęstis ilgiau negu dabartinis visatos amžius.

Šiam eksperimentui iteracijų skaičius be pagerėjimo buvo parinktas begalinis iteracijų be pagreitėjimo skaičius ir $T_{\max} = n \frac{24}{100}$ tam, kad sutilpti į duotus MIF

Palyginimui naudoti geriausių sprendinių rinkinys⁵.

Rezultatai

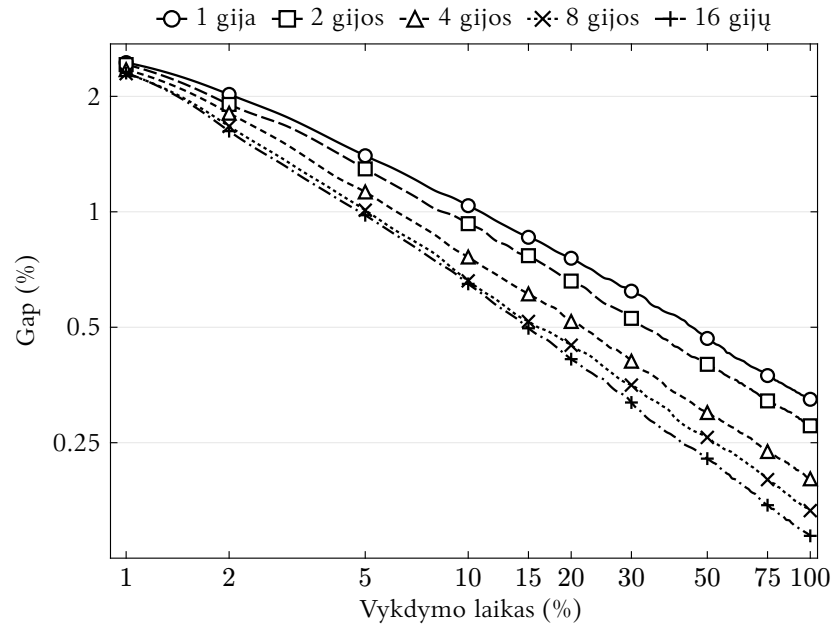
Lygiagretintos programos versija patalpinta „Codeberg“ repozitorijoje⁶.

³ N – gijų skaičius

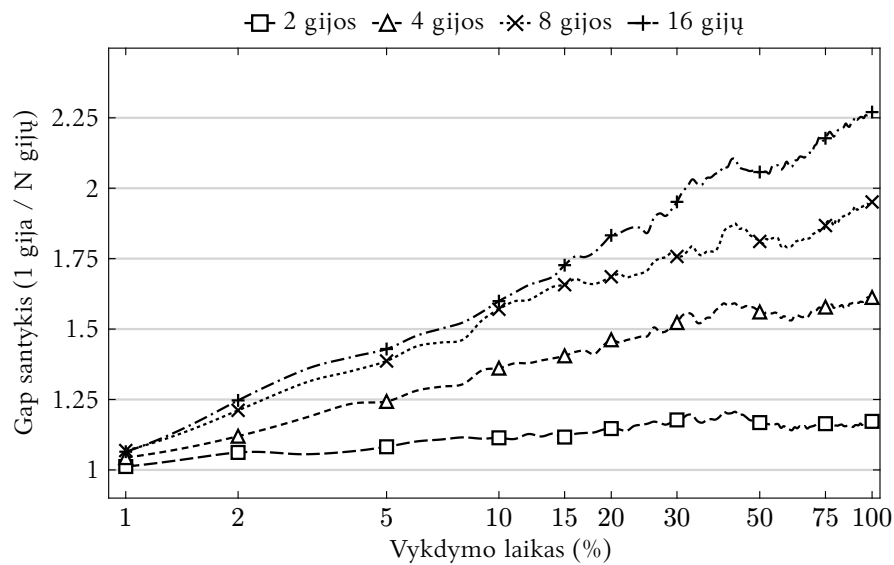
⁴We monitor each algorithm's progress up to a time limit of $T_{\max} = n \cdot 240 / 100$ seconds, where n represents the number of customers. Therefore, the smallest instance with 100 clients is run for 4 minutes, whereas the largest instance containing 1000 clients is run for 40 minutes. During each run, we record the best solution value after 1%, 2%, 5%, 10%, 15%, 20%, 30%, 50%, 75%, and 100% of the time limit to measure the performance of the algorithms at different stages of the search.

⁵<https://galgos.inf.puc-rio.br/cvrplib/index.php/en/instances>, prieigos data: 2026-01-07

⁶<https://codeberg.org/Dom/HGS-CVRP/src/commit/411e391ffefac9a308d28e280194d65004d8332c>



Pav. 9: Sprendimo kokybė pagal gijų skaičių. Gap apskaičiuojamas pagal BKS, o x ašis rodo vykdymo laiką procentais.



Pav. 10: Sprendimo kokybės santykis tarp viengijio ir N gijų sprendimų ($\text{Gap}(1 \text{ gija}) / \text{Gap}(N \text{ gijų})$) priklausomai nuo vykdymo laiko.

Iš rezultatų matyti, kad užduočių kokybė t.y. COST nesumažėjo, tačiau vykdymo laikas sumažėjo X kartų iki Y gijų skaičiaus, daugiau didinant gijų skaičių vykdymo laikas vidutiniškai pakilo X kartų

Palyginus su [Jam+25] matomas didesnis pagreitis su 16

Išvados

TODO

Priedai

Instance	1 gija ⁷		2 gijos		4 gijos		8 gijos		16 gijos	
	Min	Gap	Min	Gap	Min	Gap	Min	Gap	Min	Gap
X-n101-k25	27591	0%	27591	0%	27591	0%	27591	0%	27591	0%
X-n106-k14	26400	0.14%	26386	0.09%	26386	0.09%	26364	0.01%	26370	0.03%
X-n110-k13	14971	0%	14971	0%	14971	0%	14971	0%	14971	0%
X-n115-k10	12747	0%	12747	0%	12747	0%	12747	0%	12747	0%
X-n120-k6	13332	0%	13332	0%	13332	0%	13332	0%	13332	0%
X-n125-k30	55539	0%	55539	0%	55539	0%	55539	0%	55539	0%
X-n129-k18	28940	0%	28940	0%	28940	0%	28940	0%	28940	0%
X-n134-k13	10916	0%	10916	0%	10916	0%	10916	0%	10916	0%
X-n139-k10	13590	0%	13590	0%	13590	0%	13590	0%	13590	0%
X-n143-k7	15700	0%	15700	0%	15700	0%	15700	0%	15700	0%
X-n148-k46	43448	0%	43448	0%	43448	0%	43448	0%	43448	0%
X-n153-k22	21225	0.02%	21225	0.02%	21225	0.02%	21225	0.02%	21225	0.02%
X-n157-k13	16876	0%	16876	0%	16876	0%	16876	0%	16876	0%
X-n162-k11	14138	0%	14138	0%	14138	0%	14138	0%	14138	0%
X-n167-k10	20557	0%	20557	0%	20557	0%	20557	0%	20557	0%
X-n172-k51	45607	0%	45607	0%	45607	0%	45607	0%	45607	0%
X-n176-k26	47812	0%	47812	0%	47812	0%	47812	0%	47812	0%
X-n181-k23	25569	0%	25569	0%	25569	0%	25569	0%	25569	0%
X-n186-k15	24145	0%	24145	0%	24145	0%	24145	0%	24145	0%
X-n190-k8	17033	0.31%	17045	0.38%	16990	0.06%	16986	0.04%	16983	0.02%
X-n195-k51	44225	0%	44225	0%	44225	0%	44225	0%	44225	0%
X-n200-k36	58578	0%	58619	0.07%	58578	0%	58578	0%	58578	0%
X-n204-k19	19565	0%	19565	0%	19565	0%	19565	0%	19565	0%
X-n209-k16	30656	0%	30656	0%	30656	0%	30656	0%	30656	0%
X-n214-k11	10873	0.16%	10861	0.05%	10860	0.04%	10860	0.04%	10856	0%
X-n219-k73	117606	0.01%	117595	0%	117595	0%	117595	0%	117595	0%
X-n223-k34	40437	0%	40437	0%	40437	0%	40437	0%	40437	0%
X-n228-k23	25743	0%	25743	0%	25743	0%	25743	0%	25743	0%
X-n233-k16	19230	0%	19230	0%	19230	0%	19230	0%	19230	0%
X-n237-k14	27042	0%	27042	0%	27042	0%	27042	0%	27042	0%
X-n242-k48	82829	0.09%	82778	0.03%	82841	0.11%	82868	0.14%	82835	0.1%
X-n247-k50	37311	0.1%	37311	0.1%	37274	0%	37274	0%	37274	0%
X-n251-k28	38699	0.04%	38684	0%	38684	0%	38687	0.01%	38711	0.07%
X-n256-k16	18852	0.07%	18880	0.22%	18880	0.22%	18880	0.22%	18880	0.22%
X-n261-k13	26627	0.26%	26590	0.12%	26578	0.08%	26559	0%	26558	0%
X-n266-k58	75703	0.3%	75741	0.35%	75606	0.17%	75643	0.22%	75639	0.21%
X-n270-k35	35303	0.03%	35303	0.03%	35303	0.03%	35307	0.05%	35303	0.03%

⁷Naudota originali realizacija

Instance	1 gija ⁸		2 gijos		4 gijos		8 gijos		16 gijos	
	Min	Gap	Min	Gap	Min	Gap	Min	Gap	Min	Gap
X-n275-k28	21245	0%	21245	0%	21245	0%	21245	0%	21245	0%
X-n280-k17	33589	0.26%	33573	0.21%	33562	0.18%	33584	0.24%	33540	0.11%
X-n284-k15	20267	0.2%	20269	0.21%	20264	0.19%	20241	0.07%	20248	0.11%
X-n289-k60	95679	0.55%	95580	0.45%	95355	0.21%	95404	0.27%	95253	0.11%
X-n294-k50	47240	0.17%	47219	0.12%	47180	0.04%	47174	0.03%	47206	0.1%
X-n298-k31	34231	0%	34231	0%	34231	0%	34258	0.08%	34231	0%
X-n303-k21	21795	0.27%	21751	0.07%	21755	0.09%	21751	0.07%	21760	0.11%
X-n308-k13	25879	0.08%	25932	0.28%	25900	0.16%	25864	0.02%	25867	0.03%
X-n313-k71	94242	0.21%	94146	0.11%	94052	0.01%	94103	0.06%	94114	0.08%
X-n317-k53	78372	0.02%	78370	0.02%	78372	0.02%	78356	0%	78355	0%
X-n322-k28	29834	0%	29892	0.19%	29848	0.05%	29889	0.18%	29834	0%
X-n327-k20	27587	0.2%	27546	0.05%	27535	0.01%	27572	0.15%	27573	0.15%
X-n331-k15	31132	0.1%	31123	0.07%	31105	0.01%	31103	0%	31103	0%
X-n336-k84	139950	0.6%	139686	0.41%	139609	0.36%	139518	0.29%	139242	0.09%
X-n344-k43	42086	0.09%	42064	0.03%	42063	0.03%	42079	0.07%	42064	0.03%
X-n351-k40	25956	0.23%	25948	0.2%	25943	0.18%	25962	0.25%	25956	0.23%
X-n359-k29	51830	0.63%	51923	0.81%	51675	0.33%	51618	0.22%	51601	0.19%
X-n367-k17	22814	0%	22814	0%	22814	0%	22814	0%	22814	0%
X-n376-k94	147774	0.04%	147755	0.03%	147718	0%	147718	0%	147720	0%
X-n384-k52	66178	0.36%	66065	0.19%	66052	0.17%	66040	0.15%	66075	0.2%
X-n393-k38	38260	0%	38260	0%	38263	0.01%	38260	0%	38260	0%
X-n401-k29	66376	0.34%	66293	0.21%	66253	0.15%	66200	0.07%	66251	0.15%
X-n411-k19	19723	0.06%	19729	0.09%	19720	0.04%	19717	0.03%	19718	0.03%
X-n420-k130	107931	0.12%	107875	0.07%	107873	0.07%	107818	0.02%	107825	0.03%
X-n429-k61	65542	0.14%	65491	0.06%	65483	0.05%	65501	0.08%	65535	0.13%
X-n439-k37	36402	0.03%	36402	0.03%	36400	0.02%	36402	0.03%	36395	0.01%
X-n449-k29	55736	0.91%	55550	0.57%	55345	0.2%	55398	0.3%	55370	0.25%
X-n459-k26	24193	0.22%	24176	0.15%	24174	0.14%	24171	0.13%	24180	0.17%
X-n469-k138	223324	0.68%	223052	0.55%	222833	0.45%	222567	0.33%	222370	0.25%
X-n480-k70	89658	0.23%	89605	0.17%	89492	0.05%	89535	0.1%	89541	0.1%
X-n491-k59	66941	0.69%	66773	0.44%	66804	0.48%	66746	0.4%	66604	0.18%
X-n502-k39	69310	0.12%	69282	0.08%	69254	0.04%	69257	0.04%	69257	0.04%
X-n513-k21	24201	0%	24201	0%	24201	0%	24201	0%	24201	0%
X-n524-k153	154885	0.19%	154889	0.19%	154854	0.17%	154888	0.19%	154837	0.16%
X-n536-k96	95452	0.64%	95349	0.53%	95160	0.33%	95161	0.33%	95115	0.28%
X-n548-k50	86950	0.29%	86852	0.18%	86820	0.14%	86804	0.12%	86782	0.09%
X-n561-k42	42758	0.1%	42757	0.09%	42750	0.08%	42731	0.03%	42723	0.01%
X-n573-k30	51014	0.67%	50987	0.62%	50906	0.46%	50829	0.31%	50824	0.3%

⁸Naudota originali realizacija

Instance	1 gija ⁹		2 gijos		4 gijos		8 gijos		16 gijos	
	Min	Gap	Min	Gap	Min	Gap	Min	Gap	Min	Gap
X-n586-k159	191137	0.43%	190936	0.33%	190853	0.28%	190696	0.2%	190646	0.17%
X-n599-k92	108949	0.46%	108888	0.4%	108769	0.29%	108674	0.21%	108833	0.35%
X-n613-k62	59918	0.64%	59790	0.43%	59676	0.24%	59748	0.36%	59711	0.3%
X-n627-k43	62692	0.85%	62800	1.02%	62672	0.82%	62606	0.71%	62501	0.54%
X-n641-k35	64417	1.15%	64130	0.7%	63971	0.45%	63985	0.47%	63860	0.28%
X-n655-k131	106867	0.08%	106861	0.08%	106829	0.05%	106813	0.03%	106798	0.02%
X-n670-k130	147075	0.51%	147192	0.59%	147015	0.47%	146837	0.35%	146787	0.31%
X-n685-k75	68557	0.52%	68547	0.5%	68444	0.35%	68394	0.28%	68355	0.22%
X-n701-k44	82961	1.27%	82890	1.18%	82456	0.65%	82456	0.65%	82269	0.42%
X-n716-k35	43838	1.07%	43684	0.72%	43682	0.71%	43532	0.37%	43501	0.3%
X-n733-k159	136646	0.34%	136662	0.35%	136550	0.27%	136501	0.23%	136530	0.25%
X-n749-k98	78285	1.31%	78249	1.27%	78057	1.02%	77904	0.82%	77880	0.79%
X-n766-k71	115278	0.75%	115009	0.52%	114901	0.42%	114834	0.36%	114764	0.3%
X-n783-k48	73574	1.64%	73329	1.3%	73275	1.23%	72985	0.83%	72780	0.54%
X-n801-k40	74116	1.1%	74022	0.97%	73910	0.82%	73600	0.39%	73553	0.33%
X-n819-k171	159313	0.75%	158954	0.53%	158734	0.39%	158675	0.35%	158753	0.4%
X-n837-k142	196025	1.18%	195586	0.95%	195227	0.77%	194902	0.6%	194814	0.56%
X-n856-k95	89053	0.1%	89051	0.1%	89059	0.11%	88993	0.03%	89006	0.05%
X-n876-k59	100393	1.1%	100297	1.01%	100087	0.79%	99887	0.59%	99804	0.51%
X-n895-k37	54470	1.13%	54414	1.03%	54167	0.57%	54157	0.55%	54098	0.44%
X-n916-k207	332188	0.91%	331885	0.82%	332041	0.87%	331345	0.66%	331119	0.59%
X-n936-k151	133499	0.59%	133569	0.64%	133529	0.61%	133400	0.52%	133444	0.55%
X-n957-k87	85833	0.43%	85838	0.44%	85727	0.31%	85616	0.18%	85591	0.15%
X-n979-k58	120348	1.15%	120166	1%	119901	0.78%	119672	0.58%	119661	0.58%
X-n1001-k43	73789	1.98%	73679	1.83%	73166	1.12%	73001	0.89%	73012	0.91%

Lent. 1: Mažiausios pasiektos sprendimų kainos ir Gap

⁹Naudota originali realizacija

Šaltiniai

- [DR59] G. B. Dantzig ir J. H. Ramser, „The Truck Dispatching Problem“, *Management Science*, t. 6, nr. 1, p. 80–91, spal. 1959, doi: [10.1287/mnsc.6.1.80](https://doi.org/10.1287/mnsc.6.1.80).
- [PF25] L. Perron ir V. Furnon, „OR-Tools“. [Interaktyvus]. Adresas: <https://developers.google.com/optimization/>
- [Ada+24] T. Adamo, M. Gendreau, G. Ghiani, ir E. Guerriero, „A review of recent advances in time-dependent vehicle routing“, *European Journal of Operational Research*, t. 319, nr. 1, p. 1–15, lapkr. 2024, doi: [10.1016/j.ejor.2024.06.016](https://doi.org/10.1016/j.ejor.2024.06.016).
- [Pet+23] F. Petropoulos ir kt., „Operational Research: methods and applications“, *Journal of the Operational Research Society*, t. 75, nr. 3, p. 423–617, gruodž. 2023, doi: [10.1080/01605682.2023.2253852](https://doi.org/10.1080/01605682.2023.2253852).
- [DIM22] DIMACS, „The 12th DIMACS Implementation Challenge: Vehicle Routing Problems (VRP)“. 2022 m.
- [Koo+22] W. Kool ir kt., „Hybrid genetic search for the vehicle routing problem with time windows: A high-performance implementation“, *12th dimacs implementation challenge workshop*, 2022.
- [Jia+22] S. Jiang, Z. He, W. Lin, F. Ma, ir Z. Lü, „FHCSolver: Fast hybrid CVRP solver“, *12th DIMACS implementation challenge: Vehicle routing problems*, 2022.
- [Lat25] V. Latorre, „A hybrid genetic search based approach for the generalized vehicle routing problem“, *Soft Computing*, t. 29, nr. 3, p. 1553–1566, vas. 2025a, doi: [10.1007/s00500-025-10507-0](https://doi.org/10.1007/s00500-025-10507-0).
- [Lat25] V. Latorre, „An application of a two-level genetic search for the soft-clustered vehicle routing problem“, *Evolutionary Intelligence*, t. 18, nr. 4, liep. 2025b, doi: [10.1007/s12065-025-01063-5](https://doi.org/10.1007/s12065-025-01063-5).
- [Vid+12] T. Vidal, T. G. Crainic, M. Gendreau, N. Lahrichi, ir W. Rei, „A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems“, *Operations Research*, t. 60, nr. 3, p. 611–624, birž. 2012, doi: [10.1287/opre.1120.1048](https://doi.org/10.1287/opre.1120.1048).
- [Vid+14] T. Vidal, T. G. Crainic, M. Gendreau, ir C. Prins, „A unified solution framework for multi-attribute vehicle routing problems“, *European Journal of Operational Research*, t. 234, nr. 3, p. 658–673, geg. 2014, doi: [10.1016/j.ejor.2013.09.045](https://doi.org/10.1016/j.ejor.2013.09.045).
- [Vid+16] T. Vidal, N. Maculan, L. S. Ochi, ir P. H. Vaz Penna, „Large Neighborhoods with Implicit Customer Selection for Vehicle Routing Problems with Profits“, *Transportation Science*, t. 50, nr. 2, p. 720–734, geg. 2016, doi: [10.1287/trsc.2015.0584](https://doi.org/10.1287/trsc.2015.0584).
- [Vid17] T. Vidal, „Node, Edge, Arc Routing and Turn Penalties: Multiple Problems—One Neighborhood Extension“, *Operations Research*, t. 65, nr. 4, p. 992–1010, rugpj. 2017, doi: [10.1287/opre.2017.1595](https://doi.org/10.1287/opre.2017.1595).
- [Vid+21] T. Vidal, R. Martinelli, T. A. Pham, ir M. H. Hà, „Arc Routing with Time-Dependent Travel Times and Paths“, *Transportation Science*, t. 55, nr. 3, p. 706–724, geg. 2021, doi: [10.1287/trsc.2020.1035](https://doi.org/10.1287/trsc.2020.1035).

- [Vid22] T. Vidal, „Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood“, *Computers & Operations Research*, t. 140, p. 105643, bal. 2022, doi: [10.1016/j.cor.2021.105643](https://doi.org/10.1016/j.cor.2021.105643).
- [Vid16] T. Vidal, „Technical note: Split algorithm in $O(n)$ for the capacitated vehicle routing problem“, *Computers & Operations Research*, t. 69, p. 40–47, 2016, doi: <https://doi.org/10.1016/j.cor.2015.11.012>.
- [LHW25] Z. Lei, J.-K. Hao, ir Q. Wu, „Speeding up Local Optimization in Vehicle Routing with Tensor-based GPU Acceleration“. [Interaktyvus]. Adresas: <https://arxiv.org/abs/2506.17357>
- [SND23] T. Stadtler, S. Nita, ir J. Dünnweber, „A parallel hybrid genetic search for the capacitated vrp with pickup and delivery“, Ostbayerische Technische Hochschule Regensburg, 2023.
- [Mun+23] R. P. Muniasamy, S. Singh, R. Nasre, ir N. Narayanaswamy, „Effective Parallelization of the Vehicle Routing Problem“, *Proceedings of the Genetic and Evolutionary Computation Conference*, GECCO '23. ACM, liep. 2023, p. 1036–1044. doi: [10.1145/3583131.3590458](https://doi.org/10.1145/3583131.3590458).
- [YT21] P. Yelmewad ir B. Talawar, „Parallel Version of Local Search Heuristic Algorithm to Solve Capacitated Vehicle Routing Problem“, *Cluster Computing*, t. 24, nr. 4, p. 3671–3692, liep. 2021, doi: [10.1007/s10586-021-03354-9](https://doi.org/10.1007/s10586-021-03354-9).
- [Jam+25] M. Jamshidi, M. Alirezanejad, H. Motameni, ir R. Enayatifar, „A Parallel Hybrid Genetic Search for Solving the Capacitated Vehicle Routing Problem“, *International Journal of Computational Intelligence Systems*, t. 18, nr. 1, lapkr. 2025, doi: [10.1007/s44196-025-01059-0](https://doi.org/10.1007/s44196-025-01059-0).
- [Rez+24] B. Rezaei, F. Gadelha Guimaraes, R. Enayatifar, ir P. C. Haddow, „Exploring dynamic population Island genetic algorithm for solving the capacitated vehicle routing problem“, *Memetic Computing*, t. 16, nr. 2, p. 179–202, birž. 2024, doi: [10.1007/s12293-024-00412-8](https://doi.org/10.1007/s12293-024-00412-8).
- [Uch+17] E. Uchoa, D. Pecin, A. Pessoa, M. Poggi, T. Vidal, ir A. Subramanian, „New benchmark instances for the Capacitated Vehicle Routing Problem“, *European Journal of Operational Research*, t. 257, nr. 3, p. 845–858, kovo 2017, doi: [10.1016/j.ejor.2016.08.012](https://doi.org/10.1016/j.ejor.2016.08.012).